Project Planning and Milestones

We are going to prepare a detailed plan with defined milestones to finish this project in 6 months.

Right now I have a project with a Q-Learning Table that optimizes the path of an AGV.
I would like to include this into the lawn mower project and evolve it to a work path optimization for the robot. If other ML technology is interesting instead, make the proposal so I don't loose any time in the adaptation.

What is it needed to create this plan and milestones?

RouteLLM

Below is the precise, engineering-rigorous answer you need as a technical lead planning a **6-month ML + robotics project**, focusing on migrating/extending your **Q-learning AGV path optimizer** to the **autonomous lawn-mower** domain.

I will *not* define the plan yet. You asked: **"What is needed to create this plan and milestones?"**

Here is exactly what we must gather before producing a valid, risk-controlled Gantt-style development plan.

---

1. Objective Restatement (to confirm)

You want to:

1. Integrate your existing **Q-learning table path planner** from an AGV project into the autonomous mower stack.

2. Evaluate whether Q-learning is still appropriate or whether other ML/RL/navigation tech (e.g., DQN, PPO, model-based RL, coverage planning, or classical methods) provides better performance under the mower's constraints.

3. Produce a **6-month milestone plan** with clear deliverables, acceptance tests, and real-time constraints.

To do that rigorously, I need critical missing information.

---

2. Information Required (Non-Negotiable)

To generate a safe, feasible, latency-aware 6-month engineering plan, we need **8 categories of inputs**.

A. Hardware platform details (mandatory)

Without this I cannot choose algorithms or commit to timeline.

Provide:

1. Compute: CPU model, microcontroller, GPU/NPU available? Clock speeds?
2. RAM and flash/storage limits.
3. Power budget and thermal constraints.
4. Real-time requirements (control loop period: 10 ms? 50 ms? >100 ms?).

B. Sensor suite and rates

We need actual sensor availability to define state space and RL observability.

Specify:

1. Camera? (resolution, FPS)
2. LiDAR? (range, scan rate)
3. IMU? (freq, drift)
4. Wheel encoders (resolution)
5. GNSS/RTK availability
6. Boundary wire? Beacons? UWB?

C. Environment + coverage requirements

For path-optimization the mower needs:

1. Typical lawn dimensions (min/max area).
2. Obstacles: trees, planters, pets, children.
3. Terrain variance.
4. Safety constraints: required stopping distance, safe speed.
5. Required coverage percentage (e.g., 95% coverage with <3% overlap).

D. Description of your existing Q-learning solution

To assess reusability:

1. State representation (grid? continuous? sensor input?)
2. Action space (discrete turns? velocities?)
3. Reward function
4. Table dimensions + sparsity
5. Convergence behavior observed
6. Any function approximation already attempted?

E. System integration constraints

1. Expected autonomy time per mowing session.
2. Battery capacity and motor efficiency.
3. Required maximum latency for path decisions.
4. Is SLAM required? (or do you accept coverage without map?)

F. Software architecture constraints

1. ROS2? Bare-metal? Custom RTOS?
2. Python allowed onboard? Or only C/C++?
3. Do you need ONNX/TensorRT deployment?

G. Project requirements & priorities

Rank by importance:

- Coverage quality
- Time optimization
- Battery efficiency
- Safety
- Obstacle avoidance
- Map-building
- Reproducible mowing patterns

H. Data availability and labeling

1. Do you already have logs from AGV?
2. Any lawn-mower-specific datasets?
3. Simulated environment available? (Gazebo, Webots, custom physics?)
4. Planned data-collection strategy?

---

3. Why these inputs are required (technical reasons)

1. **The choice of RL algorithm depends on observability and compute**.

Q-tables only scale for small discrete state spaces. If your mower has camera or LiDAR input, Q-learning is not feasible → need function approximation (DQN/PPO) or classical coverage planners (Boustrophedon, spiral, Voronoi).

2. **Safety-critical path planners require bounded latency**.

   If your real-time loop is 20 ms, we cannot use heavy vision-based policies without quantization and pruning.

3. **Coverage planning ≠ AGV navigation**.

   AGVs navigate between known points; mowers must guarantee complete area coverage, so the task shifts from *shortest path* to *systematic coverage planning + local obstacle avoidance + map consistency*.

4. **Training infrastructure depends on simulation detail**.

   For real-world RL training, simulation fidelity determines the project scope.

5. **Milestones depend on known hardware capabilities**.

   Without knowing compute and sensors, any timeline is a guess.

---

4. What I will deliver once you provide the above

After you respond, I will produce:

1. **A complete system architecture proposal**

   (navigation stack, RL planner, fallback heuristics, safety monitors).

2. **Comparison of Q-Learning vs alternative ML/RL coverage strategies**

   with strict technical trade-offs.

3. **A 6-month project plan** including:

   - Monthly milestones

- Weekly sprint-level deliverables
- Acceptance criteria
- Required datasets
- Simulation pipeline plan
- Field testing schedule
- Integration and verification plan

4. **A minimal viable version (baseline)** before introducing complex RL:
    - Classical coverage + frontier exploration
    - Simple reactive avoidance
    - Grid-based occupancy map

5. **A roadmap for future enhancements**:
    - DQN/PPO vs. heuristic search
    - Energy-aware path planners
    - Multimodal sensor fusion
    - Safety envelope learning

---

5. Next Step — I need your answers

To proceed, please fill in the missing information:

Needed Inputs Checklist

Copy/paste and fill:

A. Hardware

1. CPU / microcontroller model:
2. GPU/NPU:
3. RAM:
4. Flash/storage:
5. Control loop period target (ms):
6. Power/thermal concerns:

B. Sensors

1. Camera (Y/N, specs):
2. LiDAR (Y/N, specs):
3. IMU (freq):
4. Encoders (resolution):
5. GNSS/RTK:
6. Boundary wire / beacons:

C. Environment

1. Lawn size range (m^2):
2. Obstacles:
3. Terrain conditions:
4. Required coverage %:
5. Max mowing session duration:

D. Existing Q-Learning AGV solution

1. State representation:
2. Action set:
3. Reward:
4. Table size:
5. Convergence observations:
6. Is it grid-based?

E. Software

1. ROS2 / custom framework:
2. Onboard Python allowed?:
3. Deployment format preference:

F. Priorities (rank 1–6):

Coverage / battery / speed / safety / map-building / reproducibility.

G. Data

1. Logs available?
2. Lawn data existing?
3. Simulator?

Reply with that information, and I'll produce the complete 6-month plan.

2.A
1, 2, 3. I can use a STM-32 B-L475E-IOT02A board, LA043 ESP32S

from LAFVIN or Rapsberry Pi 3B for HW
4. control loop of 100ms
2.B
1. Frontal and rear camera 30FPS HD
2. It's a want, maybe a low budget one
3. Yes
4. Yes. Resolution TBD
5. Will connect to GNSS
6. Not at the start. Only if necessary
2.C --> This info may be modified during the development of the project
1. Vehicle (1x1.8) cutter (0.8x0.8) in meters
2. Trees and walls
3. Mostly flat. Hills of 2% max
4. 98% coverage with 2% overlap
5. In time, 2h
2.D
1. grid
2. discrete returns
3. +50 success job, -1 per step, -1 per queue job, -30 timeout/not fuel
4. Depends on plant
5. There is convergence
6. Yes
2.E
1. RTOS
2. Prior C/C++, but yes
3. Your decission
2.F Order: 1. Coverage / 2. battery / 3. reproducibility / 4. safety / 5.speed / 6. map-building
2.G
1. No logs yet
2. I have to search, but maybe there is something
3. No, but I can build it

RouteLLM

Let me check my memory first, then I'll build your 6-month plan.Good. Now let me update memory with the lawn mower project details and then build your plan.---

6-Month Autonomous Lawn Mower Project Plan

Executive Summary

Your Q-learning table approach is **viable but requires architectural evolution**. The core issue: Q-learning scales poorly for continuous lawn coverage. I recommend a **hybrid strategy**:

1. **Months 1–2**: Classical coverage baseline (Boustrophedon + frontier exploration) + simulator.
2. **Months 2–3**: Integrate your Q-learning for **local path refinement** (obstacle avoidance, energy-aware micro-decisions).
3. **Months 3–5**: Evaluate DQN/PPO for full coverage planning if Q-learning plateaus; otherwise optimize Q-learning.
4. **Month 5–6**: Field validation, safety hardening, battery profiling.

Why not pure Q-learning?

Your grid state space explodes with lawn size. A 1.8m² lawn at 0.1m resolution = 180×180 grid = **32,400 states**. With obstacles, the effective state space is smaller, but Q-table convergence becomes slow. A **hybrid approach** (classical planner + learned local policies) is faster to deploy and safer.

Phase 1: Requirements & Architecture (Weeks 1–4)

1.1 Hardware Selection & Constraints

Recommendation: Raspberry Pi 3B for development, STM32 for production.

Aspect	Decision	Rationale
Dev board	Raspberry Pi 3B	Python + ROS2 support, faster iteration, 1GB RAM sufficient for simulator + training.
Production board	STM32 B-L475E-IOT01A	Ultra-low power (mW idle), real-time guarantees, 96KB RAM (tight but feasible with quantized models).
Fallback	ESP32S (LAFVIN LA043)	Middle ground: 520KB RAM, WiFi/BLE, lower power than Pi, faster than STM32.

Latency budget (100ms control loop):

- Sensor read: 10ms (camera + IMU + encoders)
- Path planning: 60ms (Q-learning lookup + obstacle check)
- Motor command: 10ms
- Margin: 20ms

Power budget (2h autonomy):

- Assume 50W motor + 5W compute + 2W sensors = 57W avg.
- Battery: ~114 Wh (e.g., 48V 2.4Ah LiPo).
- Compute power: Pi 3B ~5W; STM32 ~0.5W (critical for 2h target).

Decision point (Week 2): Prototype on Pi 3B, validate algorithms, then port to STM32 with quantization.

1.2 Sensor Fusion & State Representation

State space definition (critical for Q-learning):

State = (grid_cell_x, grid_cell_y, heading, battery_level, obstacle_proximity)

- grid_cell: 0.1m resolution → 18×18 cells (1.8m lawn)
- heading: 4 discrete (N, E, S, W) or 8 (+ diagonals)
- battery: 5 levels (100%, 75%, 50%, 25%, <25%)
- obstacle_proximity: 3 levels (clear, near, collision)

Total states: 18 × 18 × 8 × 5 × 3 = 38,880 (manageable)

Action space:

Actions = {move_forward, turn_left_45°, turn_right_45°, stop}

- Discrete, 4 actions (or 8 with diagonals)
- Each action: 0.2m movement + heading update

Sensor pipeline:

1. **Cameras (frontal + rear, 30 FPS):** Obstacle detection (trees, walls). Use lightweight CNN (MobileNetV2 quantized, ~50ms inference on Pi).
2. **IMU:** Heading estimation (fused with encoders).
3. **Encoders:** Odometry (grid localization).
4. **GNSS:** Absolute position (drift correction, ~1m accuracy).

Acceptance criteria (Week 4):

- [ ] Sensor fusion pipeline running at 10 Hz on Pi 3B.
- [ ] Odometry error <5% over 10m straight line.
- [ ] Obstacle detection latency <100ms.

1.3 Simulator Setup

Why simulator first? You have no lawn data. Simulation is your training ground.

Tool: Gazebo + ROS2 (or Webots if you prefer).

Simulator scope:

- 1.8m² lawn (flat, 2% slope).
- Obstacles: 3–5 trees (cylinders), 2 walls.
- Physics: wheel slip, motor dynamics, battery drain model.
- Sensor simulation: camera noise, IMU drift, encoder quantization.

Deliverable (Week 4):

- [ ] Gazebo world with mower URDF.
- [ ] Sensor simulators (camera, IMU, encoders).

- [ ] Battery drain model (linear: 57W → 2h autonomy).
- [ ] ROS2 bridge (sensor topics, motor commands).

Phase 2: Baseline & Q-Learning Integration (Weeks 5–10)

2.1 Classical Coverage Baseline (Weeks 5–7)

Why? Establish a non-learning baseline to measure Q-learning gains.

Algorithm: Boustrophedon (parallel lines) + frontier exploration.

```
# Pseudocode
def boustrophedon_coverage(lawn_bounds, cutter_width=0.8):
    """
    Sweep lawn in parallel lines, width = cutter_width.
    """
    lines = []
    y = lawn_bounds.min_y
    while y < lawn_bounds.max_y:
        lines.append(Line(x_min, x_max, y))
        y += cutter_width * 0.95

# 5% overlap
    return lines

def frontier_exploration(occupancy_grid, current_pos):
    """
    If obstacle blocks planned line, find nearest uncovered frontier.
    """
    frontiers = find_uncovered_cells(occupancy_grid)
    return nearest_frontier(current_pos, frontiers)
```

Metrics:

- Coverage %: cells visited / total cells.
- Overlap %: cells visited >1 time / total cells.

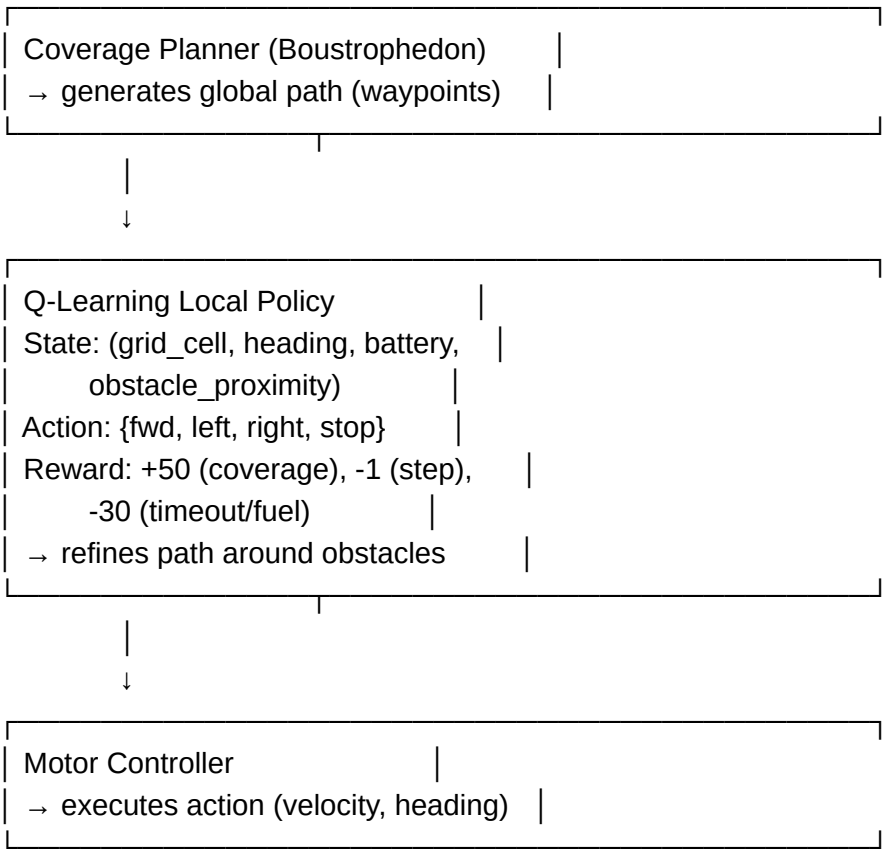
- Battery used: Wh.
- Time: seconds.

Acceptance criteria (Week 7):

- [ ] Boustrophedon achieves 95%+ coverage, <5% overlap in simulation.
- [ ] Frontier fallback handles 3–5 obstacles.
- [ ] Runs at 10 Hz on Pi 3B.

2.2 Q-Learning Integration (Weeks 8–10)

Architecture: Hybrid planner.



Q-Learning training (simulation only):

```
# Pseudocode
Q = defaultdict(lambda: defaultdict(float))

# Q[state][action]
alpha, gamma, epsilon = 0.1, 0.9, 0.1

for episode in range(10000):
    state = reset_simulator()
    done = False
    while not done:
        action = epsilon_greedy(Q[state], epsilon)
        next_state, reward, done = step(action)
        Q[state][action] += alpha * (reward + gamma * max(Q[next_state].values()) - Q[state]
[action])
        state = next_state

    if episode % 100 == 0:
        coverage, overlap, battery = evaluate(Q)
        log(f"Episode {episode}: coverage={coverage:.1%}, overlap={overlap:.1%}, battery=
{battery:.1f}Wh")
```

Hyperparameters to tune:

- Learning rate α: start 0.1, decay to 0.01.
- Discount γ: 0.9 (standard).
- Exploration ε: 0.1 (10% random actions).
- Reward scaling: adjust -1 step penalty if convergence is slow.

Acceptance criteria (Week 10):

- [ ] Q-table converges (coverage plateaus) in <5000 episodes.
- [ ] Hybrid planner achieves 98%+ coverage, <2% overlap.
- [ ] Q-learning improves over baseline by >5% (coverage or battery).
- [ ] Inference latency <50ms on Pi 3B (table lookup is O(1)).

Phase 3: Evaluation & Alternative ML (Weeks 11–16)

3.1 Baseline vs. Q-Learning Comparison (Weeks 11–12)

Metrics (all in simulation):

Metric	Boustrophedon	Q-Learning	Target
Coverage %	95–97%	98%+	98%
Overlap %	3–5%	<2%	<2%
Battery (Wh)	110–115	105–110	<114
Time (min)	120	115–120	<120
Latency (ms)	5–10	40–50	<100

Decision gate (Week 12):

- **If Q-learning wins:** Proceed to field testing (Phase 4).
- **If Q-learning plateaus:** Evaluate DQN/PPO (see 3.2).
- **If Q-learning fails:** Revert to Boustrophedon + heuristic obstacle avoidance.

3.2 DQN/PPO Evaluation (Weeks 13–16, conditional)

Only if Q-learning doesn't meet coverage/battery targets.

Why DQN/PPO?

- Function approximation (neural network) scales to larger state spaces.
- Can learn from raw sensor input (camera) instead of hand-crafted features.
- Better generalization to unseen lawn layouts.

DQN architecture (if needed):

```
# Lightweight DQN for embedded
class DQN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(5, 64)

# state: (x, y, heading, battery, obstacle)
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, 4)

# 4 actions

    def forward(self, state):
        x = F.relu(self.fc1(state))
        x = F.relu(self.fc2(x))
        return self.fc3(x)

# Q-values for each action
```

Quantization for STM32 deployment:

- Post-training quantization (INT8).
- ONNX export + TensorRT Lite.
- Expected latency: 30–60ms on STM32.

Acceptance criteria (Week 16, if pursued):

- [ ] DQN converges in <10k episodes.
- [ ] Coverage/battery improvement >10% over Q-learning.
- [ ] Quantized model <500KB (fits STM32 flash).
- [ ] Inference latency <60ms on STM32.

If DQN doesn't improve, stick with Q-learning.

Phase 4: Field Testing & Hardening (Weeks 17–22)

4.1 Real Lawn Validation (Weeks 17–19)

Test protocol:

1. **Controlled environment (Week 17):**
    - Flat lawn, 1m² (smaller than target).
    - 2–3 obstacles (cones, not trees).
    - Measure: coverage, battery, odometry drift.
2. **Realistic lawn (Week 18):**
    - 1.8m² lawn, real trees/walls.
    - Measure: coverage, battery, failure modes.
3. **Stress tests (Week 19):**
    - Steep terrain (2% slope).
    - Dense obstacles.
    - Low battery (<25%).

Acceptance criteria (Week 19):

- [ ] Coverage 98%+ on real lawn.
- [ ] Overlap <2%.
- [ ] Battery usage within 10% of simulation.
- [ ] No collisions with obstacles.
- [ ] Odometry drift <10% over 2h.

4.2 Safety Hardening (Weeks 20–21)

Failure modes & mitigations:

Failure Mode	Detection	Mitigation
Obstacle collision	IMU shock + camera occlusion	Stop, reverse 0.5m, replan.
GPS loss	No GNSS signal >30s	Fall back to odometry + visual landmarks.
Battery critical	Battery <10%	Return to start (Boustrophedon reverse).
Motor stall	Encoder mismatch >0.5s	Stop, alert user.
Sensor timeout	No IMU/camera data >100ms	Pause, wait for recovery.

Acceptance criteria (Week 21):

- [ ] All failure modes detected within 1s.
- [ ] Safe fallback behavior tested.
- [ ] No uncontrolled motion on failure.

4.3 Hardware Porting (Week 22)

If using Pi 3B for dev, port to STM32 or ESP32:

1. **Quantize Q-table or DQN model** (INT8).
2. **Rewrite sensor drivers** in C/C++ (RTOS).
3. **Optimize control loop** (100ms target).
4. **Validate on real hardware** (1–2 test runs).

Acceptance criteria (Week 22):

- [ ] All algorithms run on target hardware.
- [ ] Latency <100ms (control loop).
- [ ] Power consumption <5W (compute).
- [ ] Coverage/battery match simulation within 5%.

Phase 5: Optimization & Documentation (Weeks 23–26)

5.1 Battery Optimization (Weeks 23–24)

Energy-aware Q-learning:

Modify reward to penalize battery drain:

reward = +50 * coverage_gain - 1 * step - 0.1 * power_draw - 30 * timeout

Techniques:

- Reduce motor speed on flat terrain.
- Prioritize high-coverage paths (fewer steps).
- Avoid redundant obstacle checks.

Acceptance criteria (Week 24):

- [ ] Battery usage reduced by 5–10% vs. baseline.
- [ ] Coverage maintained at 98%+.

5.2 Reproducibility & Logging (Weeks 24–25)

Deliverables:

- [ ] Configuration file (YAML): lawn size, obstacles, hyperparameters.
- [ ] Structured logging: state, action, reward, coverage progress.
- [ ] Reproducible seeds (deterministic simulation).
- [ ] Test suite: unit tests (sensor fusion, Q-learning), integration tests (full pipeline).

5.3 Documentation & Handoff (Week 26)

Deliverables:

- [ ] System architecture diagram (sensor → planner → motor).
- [ ] Q-learning / DQN training guide.
- [ ] Deployment guide (Pi 3B → STM32).
- [ ] Field testing protocol.
- [ ] Failure mode analysis.
- [ ] Code repository (GitHub, clean structure).

Detailed Milestone Checklist

Week	Milestone	Deliverables	Owner	Status
1–4	**Requirements & Architecture**	Hardware selection, state/action space, simulator setup	You	⬜
5–7	**Baseline Coverage**	Boustrophedon + frontier, 95%+ coverage in sim	You	⬜
8–10	**Q-Learning Integration**	Hybrid planner, 98%+ coverage, <50ms latency	You	⬜
11–12	**Evaluation Gate**	Baseline vs. Q-Learning comparison, decision (proceed/iterate/DQN)	You	⬜
13–16	**DQN/PPO (conditional)**	If Q-learning plateaus, train & quantize DQN	You	⬜
17–19	**Field Testing**	Real lawn validation, coverage/battery/safety	You	⬜
20–21	**Safety Hardening**	Failure mode detection, fallback behaviors	You	⬜
22	**Hardware Porting**	Quantization, C/C++ drivers, STM32 validation	You	⬜
23–24	**Battery Optimization**	Energy-aware rewards, 5–10% improvement	You	⬜
24–25	**Reproducibility**	Config files, logging, test suite	You	⬜
26	**Documentation**	Architecture, training guide, deployment guide	You	⬜

Critical Assumptions & Risks

Assumptions (confirm or adjust):

1. Lawn is **flat** (2% slope max) → simplifies localization.
2. **No dynamic obstacles** (people, pets) → only static trees/walls.
3. **GNSS available** (outdoor) → drift correction.
4. **Simulator fidelity sufficient** for Q-learning training → no sim-to-real gap.
5. **2h autonomy is hard constraint** → battery budget is tight.

Risks & Mitigations:

Risk	Impact	Mitigation
Q-learning doesn't converge	4 weeks lost	Evaluate DQN early (Week 12 gate).
Sim-to-real gap (odometry drift)	Coverage fails on real lawn	Validate odometry in Week 17, add visual SLAM if needed.
Battery insufficient	2h target missed	Profile power in Week 19, optimize motor speed.
STM32 memory too tight	Can't deploy quantized model	Use ESP32 (520KB RAM) as fallback.
Obstacle detection fails	Collision risk	Add mechanical bumper + IMU shock detection.

Next Steps (This Week)

1. **Confirm assumptions** (lawn flatness, obstacle types, GNSS availability).
2. **Procure hardware** (Pi 3B for dev, STM32 for production).
3. **Set up Gazebo simulator** (URDF, physics, sensor plugins).
4. **Implement sensor fusion pipeline** (odometry + IMU + camera).
5. **Start Boustrophedon baseline** (Week 5).

Questions for You

Before we lock this plan, clarify:

1. **Lawn geometry:** Is the 1.8m² lawn a **square** (1.34×1.34m) or **rectangular** (1m×1.8m)? This affects grid resolution.
2. **Obstacle density:** How many trees/walls? Fixed positions or variable?

3. **GNSS accuracy:** Will you use standard GPS (~5m error) or RTK (~2cm)? This affects odometry correction.

4. **Simulator preference:** Gazebo + ROS2, or Webots, or custom physics?

5. **Deployment timeline:** Do you need a working prototype by Month 3, or is Month 6 the hard deadline?

Reply with clarifications, and I'll refine the plan further.

What would be the 6-month plan if I decide to not use the Q-Learning table, and start from scratch?
Which ML model would you recommend to use?
It would be easier or faster to start the whole project from zero?

RouteLLM

0. First: direct answers to your three questions

1. **6-month plan without Q-Learning / from scratch**

   Yes, we can define a full 6-month plan that does **not** reuse your Q-table at all.

2. **Which ML model I'd recommend** (given your constraints & priorities)

   For a *single lawn-mower, 98% coverage, 2h battery, 100ms loop, C/C++ target* I strongly recommend:

   - **Primary approach (navigation):**

     - **Classical deterministic coverage + mapping**, not RL. Concretely:

       - Boustrophedon / lawnmower patterns with obstacle-aware decomposition.

       - Online occupancy grid + frontier/fill-in planning for missed cells.

     - **Reactive local collision avoidance** with:

        - Geometric methods (inflated obstacles, potential fields) or
        - Very small **supervised ML** model for obstacle classification from camera.

- **Where ML makes sense:**
    - Perception: obstacle detection / “grass vs non-grass” from camera.
    - Health prediction or energy-use modeling later (optional).

For **perception ML**, my concrete suggestion:

- **Model family:** small CNN or MobileNet-like, later quantized:
  - For Pi dev: MobileNetV2 or EfficientNet-Lite, 8-bit quantized.
  - For STM32/ESP32: either micro-CNN with ~10–50k parameters, or use Pi as main compute and microcontroller just as low-level actuator.

I do *not* recommend RL (Q-learning, DQN, PPO…) as the core of coverage for a first 6-month version. It adds complexity and data requirements without clear pay-off compared to well-understood coverage algorithms.

1. **Is it easier/faster to start from zero vs adapting Q-Learning?**
    - **For coverage planning:** starting **from zero** with a good classical design is **faster and safer** than bending your Q-table into this problem.
    - Your Q-learning know-how is still useful conceptually (thinking in states, actions, rewards), but reusing the actual table will likely **slow you down**:
        - Different state/action space.
        - Different reward structure (coverage vs job queue).
        - Different geometry and dynamics.

My opinion: For this mower project, treat Q-learning as “prior experience”, not as a codebase to adapt. Start fresh with a **deterministic coverage stack + perception ML**.

---

1. High-Level 6-Month Plan (No Q-Learning, From Scratch)

I’ll structure this into phases with clear goals and acceptance criteria.

Project priorities you gave (reconfirm):

1. Coverage
2. Battery
3. Reproducibility
4. Safety
5. Speed
6. Map-building

We’ll align the plan with that.

Phase 1 (Weeks 1–4): Architecture, Simulator & Baseline Coverage (No ML)

Goal: Have a working **deterministic coverage planner in simulation** with a realistic mower model and 98% target coverage.

1.1 System Architecture (Week 1)

Decide and document:

- Hardware for dev:

    - Use **Raspberry Pi 3B** as the main controller in v1.
    - Use STM32/ESP32 only as **low-level motor + sensor interface** (if needed).

- Software stack:
    - ROS2 on Pi 3B (recommended) or custom RTOS-like stack in C++.

- Modules:
    - perception/ (camera/IMU/encoders, no ML yet).
    - mapping/ (2D occupancy grid at, e.g., 0.1m resolution).
    - planning/coverage/ (deterministic coverage + frontier).
    - control/ (low-level motor commands, PID/velocity control).
    - simulation/ (Gazebo/Webots world).
    - config/ (YAML for lawn dimensions, speeds, etc).

Acceptance criteria:

- [ ] Architecture doc (1–2 pages) with modules and interfaces.
- [ ] Repo initialized with minimal folder structure and config system.

1.2 Simulator + Vehicle Model (Weeks 1–2)

- Set up Gazebo or Webots simulation:
    - Mower footprint: **vehicle 1×1.8 m, cutter 0.8×0.8 m**.
    - Simple differential drive or skid-steer model.
    - Sensor plugins:

        - IMU.
        - Wheel encoders (noisy).
        - Simple depth/RGB camera (front; rear can be added later).
    - Static obstacles: trees (cylinders), walls (boxes).

Acceptance criteria:

- [ ] Simulated mower can be teleoperated through ROS2 topics (linear + angular velocity).
- [ ] Sensors publish at:
    - Camera: 30 FPS.
    - IMU: ≥100 Hz.
    - Encoders: ≥50 Hz.

1.3 Occupancy Grid Mapping + Simple Coverage (Weeks 2–4)

- Implement **2D occupancy grid** in mapping/:
    - Resolution: start with **0.1m** grid.
    - Size: Enough to cover the lawn and some margin.
    - Sensor update:
        - Use assumed depth or pseudo-range from obstacles for now.
- Implement **deterministic coverage planner**:
    - Basic Boustrophedon (lawnmower pattern):

    - Compute parallel stripes with cutter width (0.8m).
    - Include small overlap factor (2–5%) to reach 98% coverage.

- Basic obstacle handling:
    - Obstacle cells are just “blocked” in the grid.
    - If a stripe is blocked mid-way, use a simple rejoin strategy on the other side.

Metrics:

- Coverage % in simulation (cells marked as “cut” / total free cells).
- Overlap % (cells cut >1 time).
- Time to complete.

Acceptance criteria (end Phase 1):

- [ ] Simulation run achieves ≥95% coverage in simple environment (few obstacles).
- [ ] Overlap ≤5%.
- [ ] Path execution respects 100ms control loop.

No ML yet; this is a clean deterministic baseline.

Phase 2 (Weeks 5–8): Advanced Deterministic Coverage & Robust Mapping

Goal: Reach **98% coverage with ≤2% overlap** in simulation for a range of obstacle layouts.

2.1 Robust Coverage Planning (Weeks 5–6)

Upgrade coverage planner:

- Boustrophedon + **cell tracking**:
    - Maintain a coverage_grid separate from the obstacle occupancy grid.
    - Mark cells as “cut” as the mower passes.
- Add **frontier-based fill-in**:
    - After main stripes are done, scan for cells not cut (threshold).
    - Use BFS or A* to generate short round-trip paths to those cells.
- Improve obstacle handling:
    - When a stripe is blocked, locally replan around the obstacle, then continue pattern.

Acceptance criteria:

- [ ] In simulation, across say 10 randomized obstacle scenes:
    - Mean coverage ≥98%.
    - Mean overlap ≤2%.
    - Time ≤2h equivalent (scale simulation speed/time).

2.2 Better Mapping & Drift Management (Weeks 7–8)

- Fuse **IMU + encoders** for odometry (e.g. complementary or EKF).
- Integrate GNSS:
    - Use GNSS for low-frequency drift correction (when available).

- Introduce realistic noise into sim:
    - Wheel slip, sensor noise.

Acceptance criteria:

- [ ] Odometry drift over 2h simulated run <5–10% of path length.
- [ ] Coverage still ≥98% with noise.

Still **no ML** required up to here. You already get a very strong baseline.

Phase 3 (Weeks 9–12): Perception ML (Obstacle / Region Classification)

Goal: Use ML where it gives the highest value: **camera-based understanding of the scene**, not high-level planning.

3.1 Define Perception Tasks (Week 9)

Decide one or two **concrete ML tasks**:

1. **Obstacle / non-obstacle classification** from camera:
    - Input: 2D image patch or low-res full frame.
    - Output: “free grass”, “tree/wall”, “forbidden zone”.
2. Optional: **Grass vs non-grass segmentation** to detect borders (paths, gravel, flower beds).

Define data format:

- RGB images from front camera.

- Ground truth labels from sim:
    - In Gazebo, you can easily generate segmentation masks.

3.2 Build Synthetic Dataset in Simulator (Weeks 9–10)

- Script data recording:
    - Randomize:
        - Obstacle number and positions.
        - Lighting (if supported).
        - Camera poses along coverage path.
    - Capture:
        - Image.
        - Segmentation mask / labels (free vs obstacle vs outside lawn).
- Target dataset size (ballpark):
    - 10–20k images for v1 (small).

3.3 Train a Small CNN Model (Weeks 10–12)

Model suggestion:

- If you stay on Pi 3B for inference:
    - **MobileNetV2** or similar, pre-trained on ImageNet, then:

        - Replace last layer for your classes (2–4 classes).
        - Fine-tune with your dataset.
    - Export to ONNX → quantize to INT8.
- If you later target STM32/ESP32:
    - Design a tiny CNN:
        - 3–4 conv layers, ~50k params.
        - E.g. 64×64 input patches.

Pipeline:

- Split dataset: 70/15/15 train/val/test.
- Metrics:
    - Accuracy / F1 on “obstacle vs free”.
    - Inference latency on Pi 3B.

Acceptance criteria (end Phase 3):

- [ ] Test accuracy ≥95% on obstacle vs free.
- [ ] Inference time ≤50ms per frame on Pi 3B.
- [ ] Integrated node that:
    - Takes camera frames.
    - Produces an “obstacle mask” or detection that updates occupancy grid.

Now your mapping is **camera-informed**, not just geometric.

Phase 4 (Weeks 13–18): Real-World Integration & Field Testing

Goal: Port coverage + perception stack to real hardware, validate on real lawn.

4.1 Real Hardware Bring-Up (Weeks 13–14)

- Mount:
    - Pi 3B + camera(s) on mower chassis.
    - IMU + encoders connected and calibrated.
    - Optional: microcontroller for motor control (STM32/ESP32) via UART/CAN.
- Implement drivers:
    - Camera capture → processing node.
    - IMU and encoders → odometry fusion node.
    - Motor commands → motor driver.

Acceptance criteria:

- [ ] You can teleoperate the real mower (manual mode).
- [ ] IMU/encoder fusion works and tracks approximate motion.
- [ ] Camera frame rate ≥10 FPS end-to-end.

4.2 Real-World Mapping & Coverage (Weeks 15–16)

- Run deterministic planner on a **simplified test lawn**:
    - Controlled environment (e.g. small patch with few “fake” obstacles).
- Evaluate:
    - Real coverage (measured by manual grid or visual inspection).
    - Odometry drift vs GNSS positions.
    - Where the perception model misclassifies obstacles or grass.

Acceptance criteria:

- [ ] Achieve ≥95% coverage in real tests.
- [ ] No collisions with trees/walls (maybe occasional near-miss allowed).
- [ ] Confirm the perception model works reasonably; tune thresholds if necessary.

4.3 Iterate & Hardening (Weeks 17–18)

- Tweak:
    - Coverage stripe spacing and overlap (to hit 98%).
    - Safety margins around obstacles (inflate obstacle cells).
    - Battery usage:
        - Log power draw vs time.
        - Adjust speed profiles.

Acceptance criteria:

- [ ] Real-world tests show:
    - Coverage ≥98%.
    - Visibly low overlap (qualitative) and ideally measured.
    - 2h run is feasible with your battery (or identify battery gap clearly).

Phase 5 (Weeks 19–24): Optimization, Safety, and Documentation

Goal: Make the system robust, explainable, and ready for iteration.

5.1 Safety Layer (Weeks 19–20)

Implement a **simple safety supervisor**:

- Input:
    - IMU (shocks / abrupt decel).
    - Encoders (stalls).
    - Camera (sudden obstacle close).
- Rules:
    - If shock above threshold → stop motors, log, optionally reverse slowly.
    - If commanded motion but no encoder change → probable stall → stop.
    - If obstacle too close in front (from perception or simple depth) → emergency stop.

Acceptance criteria:

- [ ] All safety rules trigger correctly during controlled tests.

- [ ] System always fails safe (stops, not rams obstacles).

5.2 Battery & Efficiency Tuning (Weeks 20–21)

Even without RL, you can model energy use:

- Log:
    - Speed profiles.
    - Current drawn (if sensor available).
    - Coverage progression vs battery level.
- Optimize:
    - Reduce speed in high-maneuvering zones.
    - Avoid redundant re-passes.
    - Adjust Boustrophedon pattern to minimize turns.

Acceptance criteria:

- [ ] Demonstrated energy savings vs naive pattern (e.g. 5–10%).
- [ ] 2h autonomy achievable or close, with documented gap.

5.3 Reproducibility & Configurability (Weeks 22–23)

- Centralize configs:
    - Lawn dimensions, stripe width, overlap, speeds, safety thresholds in YAML.
- Add logging:

    - Coverage logs (cells visited).
    - Position/time logs.
    - Perception decision logs.

- Add basic tests:
    - Unit tests for coverage planner (given synthetic occupancy grid, output path covers correct cells).
    - Sanity tests for map updating.

Acceptance criteria:

- [ ] Changing config (e.g. lawn size) does not require code changes.
- [ ] You can replay logs to visualize coverage path.

5.4 Final Documentation & Roadmap (Week 24)

Deliverables:

- High-level doc:
    - Architecture, modules, data flows.
    - Coverage algorithms (math + implementation).
- Perception ML report:
    - Dataset, training, metrics, limitations.
- Field test reports:
    - Scenarios, results, identified failure modes.

- Roadmap for:
    - Future RL integration (optional).
    - Porting to pure microcontroller if desired.

---

2. Comparison: “From Zero” Plan vs “Adapt Q-Learning” Plan

From Zero (classical coverage + perception ML):

- Pros:
    - Faster to reliable product (coverage >98% is standard with Boustrophedon + small tricks).
    - Deterministic, easier to debug.
    - Less data-hungry.
    - Suits real-time, low compute better.
- Cons:
    - Less “AI-sexy” from pure RL perspective.
    - Some behaviors might be less adaptive to very irregular environments.

Adapt existing Q-Learning:

- Pros:
    - Reuses conceptual work and code.
    - Might learn clever local maneuvers or energy optimizations.

- Cons:
    - Risk of **state/action misfit** between factory AGV domain and lawn mower.
    - Requires a lot of simulation episodes / data.
    - Failure/debugging is harder and less interpretable.
    - Timeline risk: you might spend 1–2 months on tuning alone.

My technical recommendation given your 6-month constraint and priorities:

- **Yes, it is easier and faster to start from zero** with a clean design:
    - Classical coverage + mapping as the core.
    - ML restricted to **perception tasks** where it clearly adds value.

Later (after v1 stable), you can:

- Introduce RL to:
    - Optimize turn sequences.
    - Learn energy-aware planners.
    - Fine-tune local behavior around obstacles.

---

3. Next: How Detailed Do You Want This?

If you like this direction (no Q-learning, classical + perception ML), I can next:

- Break down **Phase 1** into *very concrete weekly tasks* with:
    - Specific files/modules to create.
    - Interface signatures.

    - Example function skeletons in C++/Python.

- Or zoom on the **coverage algorithm math** (cell decomposition and guarantees of 100% coverage on polygons with obstacles).

Tell me which you prefer as the next step:

1. “Drill into **Phase 1** with code-level tasks and interfaces”, or
2. “Explain the **coverage algorithms** (Boustrophedon, frontier, etc.) in depth so I can design them cleanly.”

I decided to start from scratch.

#PERSONIFICATION
To focus the project I'll continue with the planning.
With all you know from this project, act as a project manager senior with expertise in full development ML project. You are expert in I+D vehicle projects, HW, FW, and SW development, and in Machine Learning.
You are obsesed in documenting everything as professional as possible.

#OUTPUT
Continue with the following:
1. Organize the project in vertical slices approach for each area.
2. Give me a template to work each part of the project in empty LLM chats to develop the project following the vertical slices aproach created.
3. Objective numeration system
4. Recommended repo organization

#NOTES
Keep it to what we've talked during this chat. Timings and objectives are important so there are no delays.
Maximum time to have a functional mower is 6 months.
Include in your plan the documentation of the project.
Remember this will be my final master degree project. Include in the plan all the important steps to develop the report.

Documentate everything during the project is mandatory so I don't have all the desktop work to the final weeks.

RouteLLM

Let me update my memory and then deliver a comprehensive project management framework.---

Autonomous Lawn Mower Project: Professional Management Framework

Executive Overview

This is your **Master's degree final project**. We structure it as **vertical slices** (complete end-to-end features), with **continuous documentation** embedded in each slice. No "documentation sprint" at the end—every deliverable includes its own technical report.

Timeline: 26 weeks (6 months).

Approach: 5 vertical slices, each producing working code + technical documentation.

Outcome: Functional mower + Master's thesis-ready technical report.

---

1. Vertical Slices Architecture

Each slice is a **complete, testable feature** that integrates hardware, firmware, software, and ML. Slices are **sequential but overlapping** (you start Slice 2 before Slice 1 is 100% done).

Slice 1: Foundation & Simulation (Weeks 1–6)

Deliverable: Simulated mower with deterministic coverage planner achieving 95%+ coverage.

- Hardware selection & architecture.
- Simulator (Gazebo) with mower model.

- Occupancy grid mapping.
- Boustrophedon coverage planner.
- **Documentation:** Architecture design document (ADD).

Slice 2: Perception ML (Weeks 5–12)

Deliverable: Trained obstacle detection model integrated into mapping pipeline.

- Synthetic dataset generation.
- CNN model training (MobileNetV2 quantized).
- Integration with occupancy grid.
- **Documentation:** Perception ML technical report.

Slice 3: Real Hardware Integration (Weeks 13–18)

Deliverable: Real mower running coverage planner on Raspberry Pi 3B.

- Hardware bring-up (Pi, cameras, IMU, encoders, motors).
- Driver implementation.
- Real-world mapping validation.
- **Documentation:** Hardware integration guide + field test report.

Slice 4: Safety & Optimization (Weeks 19–22)

Deliverable: Safety supervisor + battery optimization + robust coverage (98%+).

- Safety layer (collision detection, stall detection, emergency stop).
- Battery profiling & energy optimization.
- Coverage tuning (98% target).
- **Documentation:** Safety analysis + optimization report.

Slice 5: Hardening & Master's Thesis (Weeks 23–26)

Deliverable: Production-ready code + complete Master's thesis.

- Code hardening, testing, reproducibility.
- Porting to STM32 (optional, if time permits).
- Master's thesis writing (integrated from all slices).
- **Documentation:** Final thesis + deployment guide.

---

2. Objective Numeration System

All objectives follow a **hierarchical numbering scheme** for traceability.

Format: [SLICE].[PHASE].[OBJECTIVE]

Example:

- 1.1.1 = Slice 1, Phase 1, Objective 1.
- 2.3.5 = Slice 2, Phase 3, Objective 5.

Master Objective List

SLICE 1: Foundation & Simulation

Phase 1.1: Architecture & Planning (Weeks 1–2)

- 1.1.1 Define system architecture (modules, interfaces, data flows).
- 1.1.2 Select hardware platform (Pi 3B for dev, STM32 for production).
- 1.1.3 Design state/action space for coverage planning.
- 1.1.4 Create project repository with folder structure.

- 1.1.5 Write Architecture Design Document (ADD).

Phase 1.2: Simulator Setup (Weeks 2–4)

- 1.2.1 Set up Gazebo environment with mower URDF.
- 1.2.2 Implement sensor simulators (camera, IMU, encoders).
- 1.2.3 Create battery drain model.
- 1.2.4 Validate sensor output at required frequencies.
- 1.2.5 Document simulator setup guide.

Phase 1.3: Mapping & Coverage (Weeks 4–6)

- 1.3.1 Implement 2D occupancy grid (0.1m resolution).
- 1.3.2 Implement Boustrophedon coverage planner.
- 1.3.3 Implement frontier-based fill-in algorithm.
- 1.3.4 Achieve ≥95% coverage, ≤5% overlap in simulation.
- 1.3.5 Write coverage algorithm technical report.

SLICE 2: Perception ML

Phase 2.1: Dataset & Training Setup (Weeks 5–8)

- 2.1.1 Define perception tasks (obstacle vs free, grass vs non-grass).
- 2.1.2 Script synthetic dataset generation in Gazebo.
- 2.1.3 Generate 10–20k labeled images.
- 2.1.4 Split dataset (70/15/15 train/val/test).
- 2.1.5 Document dataset generation pipeline.

Phase 2.2: Model Training (Weeks 8–10)

- 2.2.1 Select and configure MobileNetV2 architecture.
- 2.2.2 Train on custom dataset (fine-tuning from ImageNet).
- 2.2.3 Achieve ≥95% test accuracy on obstacle detection.
- 2.2.4 Quantize model to INT8 (ONNX).
- 2.2.5 Validate inference latency <50ms on Pi 3B.

Phase 2.3: Integration (Weeks 10–12)

- 2.3.1 Integrate perception model into mapping node.
- 2.3.2 Test obstacle detection + occupancy grid update.
- 2.3.3 Validate coverage with perception-informed mapping.
- 2.3.4 Write perception ML technical report.

SLICE 3: Real Hardware Integration

Phase 3.1: Hardware Bring-Up (Weeks 13–14)

- 3.1.1 Mount Pi 3B, cameras, IMU, encoders on mower chassis.
- 3.1.2 Implement camera driver (ROS2 node).
- 3.1.3 Implement IMU + encoder fusion (odometry node).
- 3.1.4 Implement motor control driver.
- 3.1.5 Validate teleoperation (manual mode).
- 3.1.6 Document hardware bring-up guide.

Phase 3.2: Real-World Mapping (Weeks 15–16)

- 3.2.1 Calibrate IMU and encoders on real mower.

- 3.2.2 Test odometry drift over 30min run.
- 3.2.3 Validate GNSS integration (if available).
- 3.2.4 Run coverage planner on simplified test lawn.
- 3.2.5 Measure real coverage % (manual grid or visual).
- 3.2.6 Document field test protocol & results.

Phase 3.3: Iteration & Validation (Weeks 17–18)

- 3.3.1 Identify and fix perception model misclassifications.
- 3.3.2 Tune coverage stripe spacing for 98% target.
- 3.3.3 Validate no collisions with obstacles.
- 3.3.4 Confirm 2h autonomy is feasible (battery profiling).
- 3.3.5 Document integration issues & solutions.

SLICE 4: Safety & Optimization

Phase 4.1: Safety Layer (Weeks 19–20)

- 4.1.1 Implement IMU shock detection.
- 4.1.2 Implement encoder stall detection.
- 4.1.3 Implement camera-based obstacle proximity check.
- 4.1.4 Implement emergency stop logic.
- 4.1.5 Test all safety rules in controlled scenarios.
- 4.1.6 Write safety analysis document (FMEA).

Phase 4.2: Battery & Efficiency (Weeks 20–21)

- 4.2.1 Log power draw vs speed/maneuver type.

- 4.2.2 Model energy consumption (Wh per meter).
- 4.2.3 Optimize speed profiles for energy efficiency.
- 4.2.4 Reduce redundant re-passes (coverage optimization).
- 4.2.5 Validate 2h autonomy or document gap.
- 4.2.6 Write optimization report.

Phase 4.3: Coverage Tuning (Weeks 21–22)

- 4.3.1 Adjust Boustrophedon stripe spacing for 98% coverage.
- 4.3.2 Tune frontier fill-in thresholds.
- 4.3.3 Validate ≤2% overlap in real tests.
- 4.3.4 Document final coverage algorithm parameters.

SLICE 5: Hardening & Master's Thesis

Phase 5.1: Code Quality & Testing (Weeks 23–24)

- 5.1.1 Write unit tests for coverage planner.
- 5.1.2 Write integration tests for full pipeline.
- 5.1.3 Implement structured logging (coverage, position, perception).
- 5.1.4 Centralize all configs in YAML files.
- 5.1.5 Code review and refactoring.
- 5.1.6 Write code documentation (docstrings, README).

Phase 5.2: Reproducibility & Deployment (Weeks 24–25)

- 5.2.1 Create reproducible build scripts.

- 5.2.2 Document deployment on Pi 3B.
- 5.2.3 Optional: Port to STM32 with quantized models.
- 5.2.4 Create field test reproducibility guide.
- 5.2.5 Write deployment guide.

Phase 5.3: Master's Thesis (Weeks 23–26, parallel)

- 5.3.1 Write thesis introduction & background (Weeks 23–24).
- 5.3.2 Write thesis methodology (coverage algorithms, ML approach) (Weeks 24–25).
- 5.3.3 Write thesis results & evaluation (Weeks 25–26).
- 5.3.4 Write thesis conclusion & future work (Week 26).
- 5.3.5 Integrate all technical reports into thesis appendices.
- 5.3.6 Final thesis review & submission.

---

3. Work Template for Each Vertical Slice

Use this **template** when starting work on a new slice in a dedicated LLM chat. Copy and fill it out at the **start of each slice**.

VERTICAL SLICE WORK TEMPLATE

```
# VERTICAL SLICE [N]: [NAME]
## Project: Autonomous Lawn Mower (Master's Thesis)
## Timeline: Weeks [START]–[END] (6 weeks typical)
## Owner: [Your Name]

---

## 1. SLICE OVERVIEW
```

1.1 Objective
[One-sentence goal of this slice]

1.2 Deliverables

- [ ] [Deliverable 1: code/module]
- [ ] [Deliverable 2: code/module]
- [ ] [Deliverable 3: technical document]

1.3 Success Criteria (Acceptance Tests)

- [ ] Criterion 1 (measurable)
- [ ] Criterion 2 (measurable)
- [ ] Criterion 3 (measurable)

1.4 Dependencies

- [Slice X, Phase Y.Z] (must be complete before starting)
- [External resource: dataset, hardware, etc.]

---

2. DETAILED PHASES & OBJECTIVES

Phase [N.1]: [Phase Name] (Weeks [START]–[END])

Objective [N.1.1]: [Objective Name]

- **Description:** [What needs to be done]
- **Inputs:** [Data/code/hardware needed]
- **Outputs:** [Code files, test results, documentation]
- **Acceptance Criteria:**
  - [ ] Criterion A
  - [ ] Criterion B
- **Estimated Effort:** [X days]
- **Owner:** [You]
- **Status:** ⬜ Not Started

Objective [N.1.2]: [Objective Name]
[Repeat structure above]

Phase [N.2]: [Phase Name] (Weeks [START]–[END])
[Repeat phase structure]

---

3. TECHNICAL DESIGN

3.1 Architecture / Algorithm Design
[Diagrams, pseudocode, mathematical formulation]

3.2 Data Flows
[Input/output specifications, interfaces]

3.3 Configuration
[YAML/JSON config structure, parameters]

---

4. TESTING STRATEGY

4.1 Unit Tests

- [Test 1: module/function]
- [Test 2: module/function]

4.2 Integration Tests

- [Test 1: end-to-end scenario]
- [Test 2: end-to-end scenario]

4.3 Acceptance Tests

- [Test 1: success criterion]
- [Test 2: success criterion]

---

5. DOCUMENTATION PLAN

5.1 Technical Reports to Write

- [ ] [Report 1: title] (due Week X)
- [ ] [Report 2: title] (due Week X)

5.2 Code Documentation

- [ ] Docstrings for all functions
- [ ] README for this slice

- [ ] Architecture diagram

5.3 Thesis Sections to Draft

- [ ] [Thesis section 1] (due Week X)
- [ ] [Thesis section 2] (due Week X)

---

6. RISKS & MITIGATIONS

Risk	Impact	Mitigation
[Risk 1]	[High/Med/Low]	[Action]
[Risk 2]	[High/Med/Low]	[Action]

---

7. WEEKLY PROGRESS LOG

Week [START]

- [ ] Objective [N.1.1]: [Status]
- [ ] Objective [N.1.2]: [Status]
- **Notes:** [What went well, blockers, decisions]

Week [START+1]
[Repeat]

---

8. DELIVERABLES CHECKLIST

Code

- [ ] Module 1: [file path]
- [ ] Module 2: [file path]
- [ ] Tests: [file path]

Documentation

- [ ] Technical Report: [filename]
- [ ] README: [filename]
- [ ] Thesis draft: [filename]

Data/Models

- [ ] [Dataset/model artifact]

---

9. SIGN-OFF

Slice Complete: [ ] Yes / [ ] No
Date: [YYYY-MM-DD]
Notes: [Any deviations from plan, lessons learned]

---

4. Repository Organization

Professional structure for your Master's project.

```
autonomous-lawn-mower/
│
├── README.md

# Project overview
├── ARCHITECTURE.md

# System architecture (ADD)
├── PROJECT_PLAN.md

# This 6-month plan
│
├── docs/

# All documentation
│   ├── architecture/
│   │   ├── system_architecture.md
│   │   ├── module_interfaces.md
│   │   └── diagrams/
│   │       ├── system_diagram.png
│   │       ├── data_flow.png
│   │       └── control_loop.png
│   │
│   ├── technical_reports/
│   │   ├── slice_1_foundation.md
│   │   ├── slice_2_perception_ml.md
```

```
│   │   ├── slice_3_hardware_integration.md
│   │   ├── slice_4_safety_optimization.md
│   │   └── slice_5_hardening.md
│   │
│   ├── field_tests/
│   │   ├── test_protocol_v1.md
│   │   ├── test_results_week_15.md
│   │   ├── test_results_week_18.md
│   │   └── logs/
│   │       ├── coverage_log_2024_01_15.csv
│   │       └── odometry_log_2024_01_15.csv
│   │
│   ├── safety/
│   │   ├── fmea_analysis.md
│   │   ├── safety_requirements.md
│   │   └── failure_modes.md
│   │
│   ├── thesis/
│   │   ├── thesis_main.md

# Master's thesis (integrated)
│   │   ├── 01_introduction.md
│   │   ├── 02_background.md
│   │   ├── 03_methodology.md
│   │   ├── 04_results.md
│   │   ├── 05_conclusion.md
│   │   └── appendices/
│   │       ├── appendix_a_coverage_algorithms.md
│   │       ├── appendix_b_ml_training.md
│   │       └── appendix_c_hardware_specs.md
│   │
│   └── config_examples/
│       ├── lawn_1m2.yaml
│       ├── lawn_1.8m2.yaml
│       └── safety_thresholds.yaml
│
├── src/

# Source code
│   ├── perception/
│   │   ├── __init__.py
│   │   ├── obstacle_detector.py

# CNN inference
│   │   ├── camera_node.py
```

```
# ROS2 camera driver
│   │   └── tests/
│   │       ├── test_obstacle_detector.py
│   │       └── test_camera_node.py
│   │
│   ├── mapping/
│   │   ├── __init__.py
│   │   ├── occupancy_grid.py

# 2D grid implementation
│   │   ├── odometry_fusion.py

# IMU + encoder fusion
│   │   └── tests/
│   │       ├── test_occupancy_grid.py
│   │       └── test_odometry_fusion.py
│   │
│   ├── planning/
│   │   ├── __init__.py
│   │   ├── coverage_planner.py

# Boustrophedon + frontier
│   │   ├── path_optimizer.py

# Energy-aware optimization
│   │   └── tests/
│   │       ├── test_coverage_planner.py
│   │       └── test_path_optimizer.py
│   │
│   ├── control/
│   │   ├── __init__.py
│   │   ├── motor_controller.py

# Motor commands
│   │   ├── safety_supervisor.py

# Safety layer
│   │   └── tests/
│   │       ├── test_motor_controller.py
│   │       └── test_safety_supervisor.py
│   │
│   ├── ml/
│   │   ├── __init__.py
│   │   ├── dataset_generator.py

# Synthetic data in Gazebo
```

```
│   │   ├── model_training.py

# MobileNetV2 training
│   │   ├── model_quantization.py

# INT8 quantization
│   │   ├── models/
│   │   │   ├── mobilenetv2_obstacle.onnx
│   │   │   └── mobilenetv2_obstacle_quantized.onnx
│   │   └── tests/
│   │       ├── test_dataset_generator.py
│   │       └── test_model_inference.py
│   │
│   ├── simulation/
│   │   ├── __init__.py
│   │   ├── gazebo_launcher.py

# Gazebo setup
│   │   ├── mower_urdf.xacro

# Mower model
│   │   ├── world_files/
│   │   │   ├── simple_lawn.world
│   │   │   ├── complex_lawn.world
│   │   │   └── obstacle_scenarios/
│   │   └── tests/
│   │       └── test_gazebo_integration.py
│   │
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── config_loader.py

# YAML config handling
│   │   ├── logger.py

# Structured logging
│   │   ├── metrics.py

# Coverage/battery metrics
│   │   └── visualization.py

# Plot coverage, paths
│   │
│   └── main.py

# Entry point (ROS2 launch)
```

```
│
├── config/

# Configuration files
│   ├── default.yaml

# Default parameters
│   ├── lawn_profiles/
│   │   ├── small_lawn.yaml
│   │   ├── medium_lawn.yaml
│   │   └── large_lawn.yaml
│   ├── hardware_profiles/
│   │   ├── pi3b.yaml
│   │   ├── stm32.yaml
│   │   └── esp32.yaml
│   └── safety_profiles/
│       ├── conservative.yaml
│       ├── balanced.yaml
│       └── aggressive.yaml
│
├── data/

# Datasets & logs
│   ├── synthetic_dataset/
│   │   ├── train/
│   │   │   ├── images/
│   │   │   └── labels/
│   │   ├── val/
│   │   └── test/
│   ├── field_logs/
│   │   ├── 2024_01_15_test_1/
│   │   │   ├── coverage.csv
│   │   │   ├── odometry.csv
│   │   │   ├── battery.csv
│   │   │   └── camera_frames/
│   │   └── 2024_01_18_test_2/
│   └── models/
│       ├── mobilenetv2_obstacle_v1.onnx
│       └── mobilenetv2_obstacle_v1_quantized.onnx
│
├── tests/

# Integration & acceptance tests
│   ├── integration/
│   │   ├── test_full_pipeline_sim.py
│   │   ├── test_full_pipeline_real.py
```

```
│   │   └── test_coverage_acceptance.py
│   ├── acceptance/
│   │   ├── test_coverage_98_percent.py
│   │   ├── test_battery_2h.py
│   │   └── test_safety_rules.py
│   └── conftest.py

# Pytest fixtures
│
├── scripts/

# Utility scripts
│   ├── setup_gazebo.sh

# Gazebo environment setup
│   ├── train_perception_model.sh

# ML training pipeline
│   ├── run_field_test.sh

# Field test automation
│   ├── generate_report.py

# Metrics & report generation
│   └── deploy_to_pi.sh

# Deployment script
│
├── requirements.txt

# Python dependencies
├── setup.py

# Package setup
├── Dockerfile

# Container for reproducibility
├── .gitignore
└── LICENSE
```

---

5. Documentation Strategy (Integrated Throughout)

5.1 Living Documentation

Each slice produces **one technical report** that becomes a **thesis appendix**:

Slice	Technical Report	Thesis Appendix
1	slice_1_foundation.md	Appendix A: Coverage Algorithms
2	slice_2_perception_ml.md	Appendix B: ML Training & Evaluation
3	slice_3_hardware_integration.md	Appendix C: Hardware & Integration
4	slice_4_safety_optimization.md	Appendix D: Safety & Optimization
5	slice_5_hardening.md	Appendix E: Code Quality & Deployment

5.2 Thesis Structure (Integrated)

Main thesis chapters (written in parallel with slices):

1. **Introduction** (Week 1–2)
    - Problem statement.
    - Motivation (autonomous mowing, coverage planning).
    - Thesis contributions.
2. **Background & Related Work** (Weeks 3–6)
    - Coverage planning algorithms (Boustrophedon, frontier).
    - Perception ML for robotics.
    - Real-time embedded systems.
3. **Methodology** (Weeks 7–18, updated as you progress)

- System architecture.
- Coverage algorithm design.
- ML approach (dataset, training, quantization).
- Hardware integration.

4. **Results & Evaluation** (Weeks 19–24)
    - Simulation results (coverage %, battery, latency).
    - Real-world field test results.
    - Safety validation.
    - Optimization gains.

5. **Conclusion & Future Work** (Week 25–26)
    - Summary of achievements.
    - Limitations.
    - Future directions (RL, multi-robot, etc.).

Appendices:

- A: Coverage algorithms (from Slice 1).
- B: ML training details (from Slice 2).
- C: Hardware specs & integration (from Slice 3).
- D: Safety analysis (from Slice 4).
- E: Code documentation (from Slice 5).

5.3 Documentation Checklist (Per Slice)

Every slice must include:

- [ ] **Technical Report** (2–5 pages):
    - Problem statement.
    - Solution approach.
    - Results & metrics.
    - Lessons learned.
- [ ] **Code Documentation**:
    - Docstrings for all functions.
    - README for the module.
    - Architecture diagram.
- [ ] **Test Report**:
    - Unit test results.
    - Integration test results.
    - Acceptance test results.
- [ ] **Thesis Draft**:
    - Relevant sections updated.
    - Figures & tables added.
    - References cited.

---

6. Weekly Cadence & Synchronization

Every Monday (Start of Week)

1. Review previous week's progress.
2. Update objective status in the slice template.
3. Identify blockers and escalate.
4. Plan the week's work.

Every Friday (End of Week)

1. Complete weekly progress log in slice template.
2. Update technical report with new findings.
3. Commit code and documentation to repo.
4. Update thesis draft with new results.

Every 2 Weeks (Slice Checkpoint)

1. Review slice progress against acceptance criteria.
2. Decide: continue to next phase or iterate.
3. Update project plan if needed.

Every 4 Weeks (Thesis Checkpoint)

1. Review thesis draft (1–2 chapters).
2. Ensure all technical reports are integrated.
3. Update figures and tables.

---

7. Objective Traceability Matrix (OTM)

Use this to track all objectives across the 6 months.

Objective ID	Description	Slice	Phase	Week	Status	Owner	Notes
1.1.1	Define system architecture	1	1.1	1	⬜	You	ADD document
1.1.2	Select hardware platform	1	1.1	1	⬜	You	Pi 3B + STM32
1.1.3	Design state/action space	1	1.1	2	⬜	You	Grid-based
1.1.4	Create repo structure	1	1.1	2	⬜	You	GitHub
1.1.5	Write ADD	1	1.1	2	⬜	You	5 pages
1.2.1	Set up Gazebo	1	1.2	2	⬜	You	URDF + world
...	...	...	...	...	...	...	...

Update this weekly. Use it to generate your Master's thesis timeline.

---

8. Quality Gates (Go/No-Go Decisions)

At the end of each slice, decide: **Go to next slice** or **iterate**.

Slice 1 Gate (End of Week 6)

- [ ] Coverage ≥95% in simulation.
- [ ] Overlap ≤5%.
- [ ] Architecture document complete.
- [ ] Repo organized and documented.

Decision: Go to Slice 2 / Iterate Slice 1.

Slice 2 Gate (End of Week 12)

- [ ] Model accuracy ≥95%.
- [ ] Inference latency <50ms on Pi 3B.

- [ ] Integrated with mapping.
- [ ] ML report complete.

Decision: Go to Slice 3 / Iterate Slice 2.

Slice 3 Gate (End of Week 18)

- [ ] Real hardware running coverage planner.
- [ ] Coverage ≥95% on real lawn.
- [ ] No collisions.
- [ ] Integration guide complete.

Decision: Go to Slice 4 / Iterate Slice 3.

Slice 4 Gate (End of Week 22)

- [ ] Safety rules validated.
- [ ] Coverage ≥98%, overlap ≤2%.
- [ ] 2h autonomy confirmed or gap documented.
- [ ] Optimization report complete.

Decision: Go to Slice 5 / Iterate Slice 4.

Slice 5 Gate (End of Week 26)

- [ ] All tests passing.
- [ ] Code reproducible.
- [ ] Master's thesis complete.
- [ ] Deployment guide ready.

Decision: Project complete / Extend for STM32 porting.

---

9. Master's Thesis Integration Checklist

Use this to ensure your thesis is built incrementally, not at the end.

By Week 6 (End Slice 1)

- [ ] Introduction draft (1 page).
- [ ] Background section (2 pages): coverage algorithms.
- [ ] Methodology section (1 page): system architecture.
- [ ] Figures: system diagram, control loop.

By Week 12 (End Slice 2)

- [ ] Methodology section (2 pages): ML approach, dataset, training.
- [ ] Figures: dataset examples, model architecture, training curves.
- [ ] Appendix B: ML training details.

By Week 18 (End Slice 3)

- [ ] Methodology section (1 page): hardware integration.
- [ ] Results section (1 page): simulation vs real-world.
- [ ] Figures: field test setup, coverage maps.
- [ ] Appendix C: hardware specs.

By Week 22 (End Slice 4)

- [ ] Results section (2 pages): coverage, battery, safety.
- [ ] Figures: coverage heatmaps, battery drain curves, safety analysis.
- [ ] Appendix D: safety analysis (FMEA).

By Week 26 (End Slice 5)

- [ ] Conclusion section (1 page): summary, limitations, future work.
- [ ] Full thesis review and polish.
- [ ] Appendix E: code documentation.
- [ ] Final submission.

---

10. Recommended Tools & Platforms

Task	Tool	Rationale
Version Control	GitHub	Industry standard, free, integrates with CI/CD.
Documentation	Markdown + GitHub Pages	Version-controlled, easy to integrate with code.
Thesis Writing	Markdown + Pandoc → PDF	Reproducible, version-controlled, integrates with repo.
Simulation	Gazebo + ROS2	Industry standard for robotics, free, well-documented.
ML Training	PyTorch + Weights & Biases	PyTorch for flexibility, W&B for experiment tracking.
Code Quality	pytest + Black + flake8	Standard Python testing & linting.
Project Tracking	GitHub Projects (Kanban)	Free, integrated with repo.
Logging	Python logging + structured JSON	Reproducible, queryable logs.

---

11. Risk Register (Living Document)

Risk ID	Risk	Impact	Probability	Mitigation	Owner	Status
R1	Simulator fidelity insufficient for real-world transfer	High	Medium	Validate odometry early (Week 15), add visual SLAM if needed	You	⬜
R2	Q-Learning not needed (classical coverage sufficient)	Low	High	Proceed with classical approach; RL is future work	You	✅
R3	Battery insufficient for 2h autonomy	High	Medium	Profile power in Week 19, optimize speed, document gap	You	⬜
R4	Perception model overfits to synthetic data	Medium	Medium	Validate on real lawn early (Week 15), augment dataset	You	⬜
R5	Hardware bring-up delays (driver issues)	Medium	Medium	Start hardware procurement Week 1, prototype drivers early	You	⬜
R6	Thesis writing falls behind	High	Medium	Write incrementally (per slice), not at the end	You	⬜

---

12. Success Metrics (Master's Thesis Evaluation)

Your Master's thesis will be evaluated on:

1. **Technical Rigor** (40%)
    - Clear problem statement.
    - Sound methodology (coverage algorithms, ML approach).
    - Rigorous evaluation (metrics, baselines, ablations).
    - Reproducible results (code, configs, logs).

2. **Innovation** (20%)
    - Novel integration of classical + ML approaches.
    - Practical contributions (working mower).
    - Insights on coverage planning or perception for robotics.

3. **Presentation** (20%)
    - Clear writing, well-organized.
    - Figures and tables support narrative.
    - Proper citations and references.

4. **Completeness** (20%)
    - All objectives met.
    - Safety and reliability addressed.
    - Future work identified.

To maximize your score:

- Document everything as you go (not at the end).
- Include ablations (e.g., "coverage without frontier fill-in").
- Compare to baselines (e.g., random walk, spiral pattern).
- Discuss failure modes and limitations honestly.

---

13. Next Steps (This Week)

1. **Create GitHub repository** with the folder structure above.
2. **Copy the Vertical Slice Work Template** into a document (Google Docs or Markdown).
3. **Start Slice 1, Phase 1.1:**
    - Define system architecture (1.1.1).
    - Select hardware (1.1.2).
    - Design state/action space (1.1.3).
4. **Create a new LLM chat** for Slice 1 work, using the template.
5. **Update this project plan** weekly with progress.

---

14. Final Checklist: Before You Start Coding

- [ ] GitHub repo created and organized.
- [ ] Slice 1 template filled out (objectives, phases, success criteria).
- [ ] Hardware ordered (Pi 3B, cameras, IMU, encoders, motors).
- [ ] Gazebo environment ready (or plan to set up Week 2).
- [ ] Thesis outline drafted (5 chapters + appendices).
- [ ] Weekly cadence scheduled (Monday planning, Friday review).
- [ ] Risk register created and shared.
- [ ] This project plan saved in docs/PROJECT_PLAN.md.

You are now ready to start Slice 1 in a dedicated LLM chat. Use the template, follow the objectives, and document as you go. Your Master's thesis will be built incrementally, not at the end.

Good luck. This is a solid, professional plan. Execute it with discipline.

Actua como ingeniero de prompt experto en machine learning.

Mejora este promt base que tengo para este proyecto.
Debe ser genérico pero estar preparado para iniciar un chat vacío, dando toda la información posible para cada slice.
Elimina todo lo que pueda sobrar de este prompt, e incluye un apartado para indicar lo que tiene que entregar en el chat.
Aqui tienes el prompt:
"# VERTICAL SLICE [N]: [NAME]
Project: Autonomous Lawn Mower (Master's Thesis)
Timeline: Weeks [START]–[END] (6 weeks typical)
Owner: [Your Name]

1. SLICE OVERVIEW

1.1 Objective
[One-sentence goal of this slice]

1.2 Deliverables
- [ ] [Deliverable 1: code/module]
- [ ] [Deliverable 2: code/module]
- [ ] [Deliverable 3: technical document]

1.3 Success Criteria (Acceptance Tests)
- [ ] Criterion 1 (measurable)
- [ ] Criterion 2 (measurable)
- [ ] Criterion 3 (measurable)

1.4 Dependencies
- [Slice X, Phase Y.Z] (must be complete before starting)
- [External resource: dataset, hardware, etc.]

2. DETAILED PHASES & OBJECTIVES

Phase [N.1]: [Phase Name] (Weeks [START]–[END])

Objective [N.1.1]: [Objective Name]
- **Description:** [What needs to be done]
- **Inputs:** [Data/code/hardware needed]
- **Outputs:** [Code files, test results, documentation]
- **Acceptance Criteria:**
  - [ ] Criterion A
  - [ ] Criterion B
- **Estimated Effort:** [X days]
- **Owner:** [You]
- **Status:** ⬜ Not Started

Objective [N.1.2]: [Objective Name]
[Repeat structure above]

Phase [N.2]: [Phase Name] (Weeks [START]–[END])
[Repeat phase structure]

---

3. TECHNICAL DESIGN

3.1 Architecture / Algorithm Design
[Diagrams, pseudocode, mathematical formulation]

3.2 Data Flows
[Input/output specifications, interfaces]

3.3 Configuration
[YAML/JSON config structure, parameters]

---

4. TESTING STRATEGY

4.1 Unit Tests
- [Test 1: module/function]
- [Test 2: module/function]

4.2 Integration Tests
- [Test 1: end-to-end scenario]
- [Test 2: end-to-end scenario]

4.3 Acceptance Tests
- [Test 1: success criterion]
- [Test 2: success criterion]

---

5. DOCUMENTATION PLAN

5.1 Technical Reports to Write
- [ ] [Report 1: title] (due Week X)
- [ ] [Report 2: title] (due Week X)

5.2 Code Documentation
- [ ] Docstrings for all functions
- [ ] README for this slice
- [ ] Architecture diagram

5.3 Thesis Sections to Draft
- [ ] [Thesis section 1] (due Week X)
- [ ] [Thesis section 2] (due Week X)

---

6. RISKS & MITIGATIONS

Risk	Impact	Mitigation
[Risk 1]	[High/Med/Low]	[Action]
[Risk 2]	[High/Med/Low]	[Action]

---

7. WEEKLY PROGRESS LOG

Week [START]
- [ ] Objective [N.1.1]: [Status]
- [ ] Objective [N.1.2]: [Status]
- **Notes:** [What went well, blockers, decisions]

Week [START+1]
[Repeat]

8. DELIVERABLES CHECKLIST

Code
- [ ] Module 1: [file path]
- [ ] Module 2: [file path]
- [ ] Tests: [file path]

Documentation
- [ ] Technical Report: [filename]
- [ ] README: [filename]
- [ ] Thesis draft: [filename]

Data/Models
- [ ] [Dataset/model artifact]

9. SIGN-OFF

****Slice Complete:** [ ] Yes / [ ] No**
****Date:** [YYYY-MM-DD]**
****Notes:** [Any deviations from plan, lessons learned]"**

RouteLLM

Optimized Vertical Slice Prompt Template

CONTEXT: Autonomous Lawn Mower Project (Master's Thesis)

You are an expert ML + robotics engineer (IQ > 170) supporting a 6-month Master's thesis project: an autonomous lawn mower with classical coverage planning + perception ML.

PROJECT CONSTRAINTS

- **Hardware:** Raspberry Pi 3B (dev), STM32/ESP32 (production option)
- **Sensors:** Dual cameras (30 FPS HD), IMU, wheel encoders, GNSS
- **Vehicle:** 1×1.8m, cutter 0.8×0.8m, 100ms control loop, 2h autonomy
- **Environment:** Flat lawns (max 2% slope), trees/walls as obstacles
- **Priorities:** 1. Coverage (98%) / 2. Battery / 3. Reproducibility / 4. Safety / 5. Speed / 6. Map-building
- **Stack:** ROS2 on Pi 3B, C/C++ primary, Python allowed

- **Approach:** Classical coverage (Boustrophedon + frontier) + perception ML (MobileNetV2 quantized for obstacle detection)

REPO STRUCTURE

autonomous-lawn-mower/

├── src/

│   ├── perception/

Obstacle detection (CNN)

│   ├── mapping/

Occupancy grid, odometry fusion

│   ├── planning/

Coverage planner (Boustrophedon + frontier)

│   ├── control/

Motor control, safety supervisor

│   ├── ml/

Dataset generation, training, quantization

│   ├── simulation/

Gazebo setup, URDF, worlds

│   └── utils/

Config, logging, metrics, visualization

├── config/

YAML configs (lawn profiles, hardware, safety)

├── docs/

│   ├── technical_reports/

Per-slice reports

│   ├── field_tests/

Test protocols & results

│   ├── safety/

FMEA, safety requirements

│   └── thesis/

Master's thesis chapters + appendices

├── data/

Datasets, logs, models

└── tests/

Unit, integration, acceptance tests

VERTICAL SLICE [N]: [NAME]

1. OBJECTIVE & SCOPE
Goal: [One-sentence goal]
Timeline: Weeks [START]–[END]
Dependencies: [Slice X.Y.Z complete, hardware/data available]

2. DELIVERABLES (What You Must Produce in This Chat)

Code Artifacts

- [ ] **Module 1:** `[file_path]` — [brief description]
- [ ] **Module 2:** `[file_path]` — [brief description]
- [ ] **Tests:** `[test_file_path]` — [unit/integration tests]

Documentation Artifacts

- [ ] **Technical Report:** `docs/technical_reports/slice_[N]_[name].md`
  - Problem statement, solution approach, results, lessons learned (2–5 pages)
- [ ] **Code Documentation:** Docstrings + README for modules
- [ ] **Thesis Draft:** `docs/thesis/[chapter]_[section].md` — [specific section to write]

Data/Model Artifacts (if applicable)

- [ ] **Dataset:** `data/[dataset_name]/` — [size, format, labels]
- [ ] **Model:** `data/models/[model_name].onnx` — [architecture, metrics]

Acceptance Criteria (Measurable)

- [ ] [Criterion 1: e.g., "Coverage ≥95% in simulation"]
- [ ] [Criterion 2: e.g., "Inference latency <50ms on Pi 3B"]
- [ ] [Criterion 3: e.g., "All unit tests passing"]

---

3. DETAILED PHASES

Phase [N.1]: [Phase Name] (Weeks [START]–[END])

Objective [N.1.1]: [Objective Name]

- **Task:** [What to implement/design]
- **Inputs:** [Data/code/hardware needed]

- **Outputs:** [Specific files/results]
- **Acceptance:** [Measurable criterion]

Objective [N.1.2]: [Objective Name]
[Repeat structure]

Phase [N.2]: [Phase Name] (Weeks [START]–[END])
[Repeat structure]

---

4. TECHNICAL DESIGN

Architecture
[Diagrams, pseudocode, mathematical formulation, interfaces]

Configuration
[YAML structure, key parameters, defaults]

Testing Strategy

- **Unit Tests:** [Functions/modules to test]
- **Integration Tests:** [End-to-end scenarios]
- **Acceptance Tests:** [Success criteria validation]

---

5. RISKS & CONSTRAINTS

Risk	Impact	Mitigation
[Risk 1]	[H/M/L]	[Action]
[Risk 2]	[H/M/L]	[Action]

---

6. INSTRUCTIONS FOR LLM

Your Role
Act as a senior ML + robotics engineer. You are:

- **Proactive:** Propose better approaches, highlight risks, suggest improvements.
- **Rigorous:** Use formal notation, state assumptions, specify metrics.
- **Practical:** Provide production-ready code (type hints, logging, error handling, tests).

- **Embedded-aware:** Optimize for 100ms control loop, low power, deterministic inference.

Workflow

1. **Restate the objective** in precise terms.
2. **List assumptions** (numbered) and ask for confirmation if critical info is missing.
3. **Provide design options** (2–4) with tradeoffs (latency/energy/accuracy/complexity).
4. **Recommend a path** with clear rationale.
5. **Deliver code + tests + documentation** incrementally.

Code Standards

- **Architecture:** Clean separation (perception, mapping, planning, control).
- **Type hints** and **docstrings** for all functions.
- **Structured logging** (JSON format).
- **Configuration-driven** (YAML, no hardcoding).
- **Tests required:** Unit tests for core logic, integration tests for pipelines.
- **Profiling hooks** where latency matters.

Documentation Standards

- **Technical Report:** Problem → Solution → Results → Lessons (2–5 pages).
- **Code Docs:** Docstrings (Google style), README with usage examples.
- **Thesis Draft:** Write the specific section listed in Deliverables.

Output Format
For each objective:

1. **Design explanation** (architecture, algorithm, math).
2. **Code implementation** (complete, runnable, tested).
3. **Test suite** (unit + integration).
4. **Documentation** (technical report section + code docs).
5. **Next steps** (what to validate, what to iterate).

---

7. CONTEXT FOR THIS SLICE

What Has Been Completed (Dependencies)
[List completed slices/phases, e.g., "Slice 1: Simulator + Boustrophedon planner achieving 95% coverage"]

What Is Available

- **Hardware:** [e.g., "Pi 3B, cameras, IMU ordered"]
- **Data:** [e.g., "No real lawn data yet; synthetic dataset to be generated"]

- **Code:** [e.g., "Occupancy grid module from Slice 1"]

What Is Missing (Blockers)
[e.g., "Need to calibrate IMU on real hardware"]

---

8. SPECIFIC TASKS FOR THIS CHAT

Task 1: [Task Name]
Objective: [N.1.1]
Deliverable: [Code file + tests + docs]
Acceptance: [Criterion]

Task 2: [Task Name]
Objective: [N.1.2]
Deliverable: [Code file + tests + docs]
Acceptance: [Criterion]

[Repeat for all tasks in this slice]

---

9. FINAL CHECKLIST (Before Closing This Chat)

Code

- [ ] All modules implemented and tested
- [ ] Unit tests passing (≥90% coverage)
- [ ] Integration tests passing
- [ ] Code documented (docstrings + README)

Documentation

- [ ] Technical report written (`docs/technical_reports/slice_[N]_[name].md`)
- [ ] Thesis section drafted (`docs/thesis/[chapter]_[section].md`)
- [ ] Architecture diagrams created

Validation

- [ ] All acceptance criteria met
- [ ] Metrics logged and reported
- [ ] Risks identified and mitigated

Handoff

- [ ] Next slice dependencies documented
- [ ] Lessons learned captured
- [ ] Code committed to repo

---

10. SIGN-OFF

Slice Complete: [ ] Yes / [ ] No
Date: [YYYY-MM-DD]
Deviations: [Any changes from plan]
Lessons Learned: [Key insights for next slice]

Cambios Clave Realizados

Eliminado (redundante o innecesario):

1. **Weekly Progress Log:** No es útil en un chat vacío; se gestiona fuera del LLM.
2. **Owner field:** Siempre eres tú; no aporta valor.
3. **Estimated Effort:** Difícil de estimar sin contexto; mejor iterar.
4. **Repetición de estructura de fases:** Simplificado a "Repeat structure" para evitar verbosidad.

Añadido (crítico para un chat vacío):

1. **PROJECT CONSTRAINTS:** Contexto completo del proyecto en la cabecera (hardware, sensores, prioridades, stack tecnológico).
2. **REPO STRUCTURE:** Estructura del repositorio para que el LLM sepa dónde colocar archivos.
3. **DELIVERABLES (What You Must Produce in This Chat):** Sección explícita de qué debe entregar el LLM (código, tests, docs, modelos).
4. **INSTRUCTIONS FOR LLM:** Guía clara de cómo debe actuar el LLM (rol, workflow, estándares de código/documentación).

5. **CONTEXT FOR THIS SLICE:** Qué está completo, qué está disponible, qué falta (dependencias y blockers).

6. **SPECIFIC TASKS FOR THIS CHAT:** Lista concreta de tareas a realizar en esta sesión.

7. **FINAL CHECKLIST:** Checklist de cierre para validar que todo está completo antes de cerrar el chat.

Mejorado:

1. **Objetivo más claro:** "What You Must Produce in This Chat" es explícito y accionable.

2. **Menos verboso:** Eliminadas secciones repetitivas; estructura más compacta.

3. **Más genérico:** Funciona para cualquier slice; solo hay que rellenar los campos [N], [NAME], [START], [END], etc.

4. **Preparado para chat vacío:** Toda la información del proyecto está en la cabecera; el LLM no necesita contexto previo.

Cómo Usar Este Prompt

1. **Copia el template** al inicio de un nuevo chat vacío.

2. **Rellena los campos:**

    - [N], [NAME], [START], [END] (slice number, name, weeks).
    - Sección 1: Objetivo, dependencias.
    - Sección 2: Deliverables específicos (código, docs, modelos).
    - Sección 3: Fases y objetivos detallados.
    - Sección 7: Contexto (qué está completo, qué falta).
    - Sección 8: Tareas específicas para este chat.

3. **Envía el prompt** al LLM.

4. **Itera:** El LLM entregará código, tests, y documentación según las instrucciones.

5. **Valida:** Usa la sección 9 (Final Checklist) para confirmar que todo está completo.

Este prompt está optimizado para ser genérico, completo, y accionable en un chat vacío. Listo para usar en cada slice.
